



Lab Number: 02

Course Code: CL2005

Course Title: Database Systems - Lab

Semester: Spring 2026

Agenda: SQL Basics (DDL, DML)

Instructor: Muhammad Mehdi

Email: muhammad.mehdi@nu.edu.pk

Office: Rafaqat Lab



Table of Contents

What is SQL?	1
1 SQL Command Categories	1
2 SQL Data Definition Language (DDL)	2
2.1 CREATE TABLE	2
2.2 DROP TABLE	2
2.3 ALTER TABLE	3
2.3.1 Add a New Column	3
2.3.2 Rename a Table	3
2.3.3 Rename a Column of a Table (New)	4
2.3.4 Drop a Column of a Table (New)	4
3 SQL Data Manipulation Language (DML)	4
3.1 INSERT	4
3.1.1 Inserting Values for All Columns	5
3.1.2 Inserting Values into Specific Columns	5
3.1.3 Inserting Multiple Rows in a Single Statement	5
3.2 DELETE	6
3.3 UPDATE	6
4 SQL Comments	7
4.1 Single-line Comment	7
4.2 Multi-line Comment	8



What is SQL?

SQL (Structured Query Language) is a domain-specific, declarative programming language used to interact with relational databases. It is commonly referred to as a **query language** because it allows users to retrieve, manipulate, and manage data by asking structured questions.

SQL serves as the standard language for creating, reading, updating, and deleting data in relational database systems.

SQL is used to:

- create databases and tables
- insert data
- update or delete data
- query and filter rows
- control permissions
- define constraints

Every RDBMS provides its own SQL engine but the core language concepts remain the same.

1 SQL Command Categories

SQL commands are generally divided into categories based on their purpose.

Main categories:

- DDL (Data Definition Language)
- DML (Data Manipulation Language)
- DQL (Data Query Language)
- TCL (Transaction Control Language)
- DCL (Data Control Language)

These SQL commands allow us to:

- define database structures
- insert, update, and delete data
- retrieve data using queries
- And more ...



2 SQL Data Definition Language (DDL)

DDL commands define and modify the structure of the database.

2.1 CREATE TABLE

CREATE TABLE is used to define a new table and its schema. After execution, the table structure becomes part of the database schema.

General Syntax:

```
CREATE TABLE table_name (  
    column_name data_type constraints,  
    column_name data_type constraints,  
    ...  
);
```

Example:

```
CREATE TABLE students (  
    id INTEGER PRIMARY KEY,  
    name TEXT NOT NULL,  
    department TEXT,  
    cgpa REAL CHECK (cgpa <= 4.0)  
);
```

2.2 DROP TABLE

DROP TABLE permanently deletes a table along with its schema and all its data.

General Syntax:

```
DROP TABLE table_name;
```

Example:

```
DROP TABLE students;
```



2.3 ALTER TABLE

ALTER TABLE modifies an existing table structure. SQLite supports limited **ALTER** operations.

2.3.1 Add a New Column

Restrictions in SQLite:

- New column must not have any of the following constraints:
 - **PRIMARY KEY**
 - **UNIQUE**
 - **FOREIGN KEY**
- Default value must be constant (cannot be a function call)

General Syntax:

```
ALTER TABLE table_name  
ADD COLUMN column_name data_type;
```

Example:

```
ALTER TABLE students  
ADD COLUMN email TEXT;
```

2.3.2 Rename a Table

General Syntax:

```
ALTER TABLE table_name  
RENAME TO new_table_name;
```

Example:

```
ALTER TABLE students  
RENAME TO university_students;
```

2.3.3 Rename a Column of a Table (New)

This feature is only supported on the versions of **SQLite ≥ 3.25.0**.



General Syntax:

```
ALTER TABLE table_name  
RENAME old_column_name TO new_column_name;
```

Example:

```
ALTER TABLE students  
RENAME name TO full_name;
```

2.3.4 Drop a Column of a Table (New)

This feature is only supported on the versions of **SQLite ≥ 3.35.0**.

General Syntax:

```
ALTER TABLE table_name  
DROP COLUMN column_name;
```

Example:

```
ALTER TABLE students  
DROP COLUMN dob;
```

3 SQL Data Manipulation Language (DML)

DML commands manipulate (insert, update, delete) the data stored in tables.

3.1 INSERT

The **INSERT** statement is used to **add new rows (records)** into a table. Each insertion operation creates one or more rows by providing values that correspond to the table's columns.

3.1.1 Inserting Values for All Columns

This form of **INSERT** is used when you want to provide values for **every column in the table**, in the **same order** as they were defined in the table schema.

General Syntax:

```
INSERT INTO table_name  
VALUES (value1, value2, ...);
```

Example:

```
INSERT INTO students  
VALUES (1, 'Ali', 'CS', 3.4);
```

3.1.2 Inserting Values into Specific Columns

This form allows you to insert values into **selected columns only**. Any column not listed will automatically receive its **DEFAULT** value or **NULL** (if no default is defined).

This method is safer and more flexible, especially when tables contain many columns.

General Syntax:

```
INSERT INTO table_name (column1, column2, ...)  
VALUES (value1, value2, ...);
```

Example:

```
INSERT INTO students (id, name, department)  
VALUES (2, 'Maham', 'IT');
```

3.1.3 Inserting Multiple Rows in a Single Statement

SQL allows inserting **multiple rows at once** using a single **INSERT** statement. This improves performance and reduces repetition.

General Syntax:

```
INSERT INTO table_name  
VALUES  
(value1, value2, ...),  
(value1, value2, ...),  
...;
```

Example:

```
INSERT INTO students
VALUES
(1, 'Ali', 'CS', 3.4),
(2, 'Usman', 'SE', 3.33);
```

3.2 DELETE

The **DELETE** statement is used to **remove existing rows** from a table based on a condition.

A **DELETE** statement without a **WHERE** clause will delete all rows from the table, but the table structure remains intact.

General Syntax:

```
DELETE FROM table_name
WHERE condition;
```

Examples:

```
-- Deleting One Rows
DELETE FROM students
WHERE id = 1;

-- Deleting All Rows
DELETE FROM students;
```

3.3 UPDATE

The **UPDATE** statement is used to **modify existing records** in a table. It changes the values of one or more columns for rows that satisfy a specified condition.

An **UPDATE** statement without a **WHERE** clause will update all rows of the table.

General Syntax:

```
UPDATE table_name
SET column1 = value1,
    column2 = value2,
```



```
...  
WHERE condition;
```

Examples:

```
-- Updating Single Column  
UPDATE students  
SET cgpa = 3.8  
WHERE id = 2;  
  
-- Updating Multiple Columns  
UPDATE students  
SET cgpa = 3.6,  
    department = 'CS'  
WHERE id = 3;
```

4 SQL Comments

Comments serve as statement/query explanations for developers. These are ignored by the DBMS while command execution.

4.1 Single-line Comment

Example:

```
-- This is a single-line comment  
SELECT * FROM users; -- inline comment
```

4.2 Multi-line Comment

Example:

```
/* This is  
   a multi-line  
   comment */
```



```
SELECT * FROM users;
```